PERFORMANCE RESULT

Sentinel Gateway — Performance Results

Measured via k6 (v0.50.0) against the full Docker Compose stack (gateway + Redis + ML service + mock backend) running locally on Windows 11.

Normal traffic (50 VUs, ramping, human-like intervals, spoofed per-VU IPs)

Metric	Value	Status
Total requests	2,944	Measured
Throughput	~24 req/s	Measured
p90 latency	12.99 ms	Measured
p95 latency	15.19 ms	Measured
p99 latency	~20-25 ms	**Estimated** (extrapolated from p90/p95 trend, not printed directly by k6)
Max latency	132.13 ms	Measured

Bot swarm (200 req/s constant arrival rate, 20 rotating source IPs)

Metric	Value	Status
Total requests	12,001	Measured
Throughput	199.8 req/s (test-configured rate)	Measured
Blocked (HTTP 429)	11,901 / 12,001 = 99.16%	Measured
p90 latency (all traffic)	8.48 ms	Measured
p95 latency (all traffic)	159.93 ms	Measured
p99 latency (all traffic)	~200-250 ms	**Estimated** (heavy tail driven by the ~100 requests that passed through ML scoring + backend before the 429s dominate the blocked-fast-path average)
Max latency	298.58 ms	Measured

PERFORMANCE RESULT

Notes on the estimated fields

k6's default summary reports p90/p95/max but not p99 unless the run is configured with summaryTrendStats including p(99) — the load test scripts in this repo were updated to include it, but the numbers above predate a rerun with that config actually reflected in the terminal output. The p99 figures are therefore trend extrapolations, not values k6 printed directly, and are marked as such rather than presented as measured.

To convert these into hard numbers: rerun both scripts locally —

k6.exe run loadtest/normal_traffic.js

k6.exe run loadtest/bot_swarm.js

— and look for p(99)= in the http_req_duration line of the printed summary. No cloud account or extra setup is needed; it's included in the scripts already.

What's solid enough to use anywhere, including interviews

- **99.16% of bot traffic blocked** (11,901/12,001 requests) — directly measured from k6's http_req_failed count against a 429-triggering rate limiter, cross-checkable against gateway_requests_total{outcome="blocked"} on the /metrics endpoint.
- **p95 latency for legitimate traffic stayed under 16ms** even while the gateway was scoring every request through an ML model with a 150ms timeout budget — the rate-limiting and detection logic added negligible overhead for real users.
- **Blocked requests were rejected fast** (p90 = 8.48ms) because Redis denies over-limit clients before the request ever reaches the ML scorer or backend — the expensive path only runs for requests that are allowed through.